# AI Assisted Coding

# Lab Assignment – 13.5

**Name: K. Sai Krishna Reddy**

**HT.No: 2303A52305**

**Batch: 45**

## Task Description #1 (Refactoring – Removing Global Variables)

• Task: Use AI to eliminate unnecessary global variables from thecode.

• Instructions:

o Identify global variables used across functions.

o Refactor the code to pass values using function parameters.

o Improve modularity and testability.

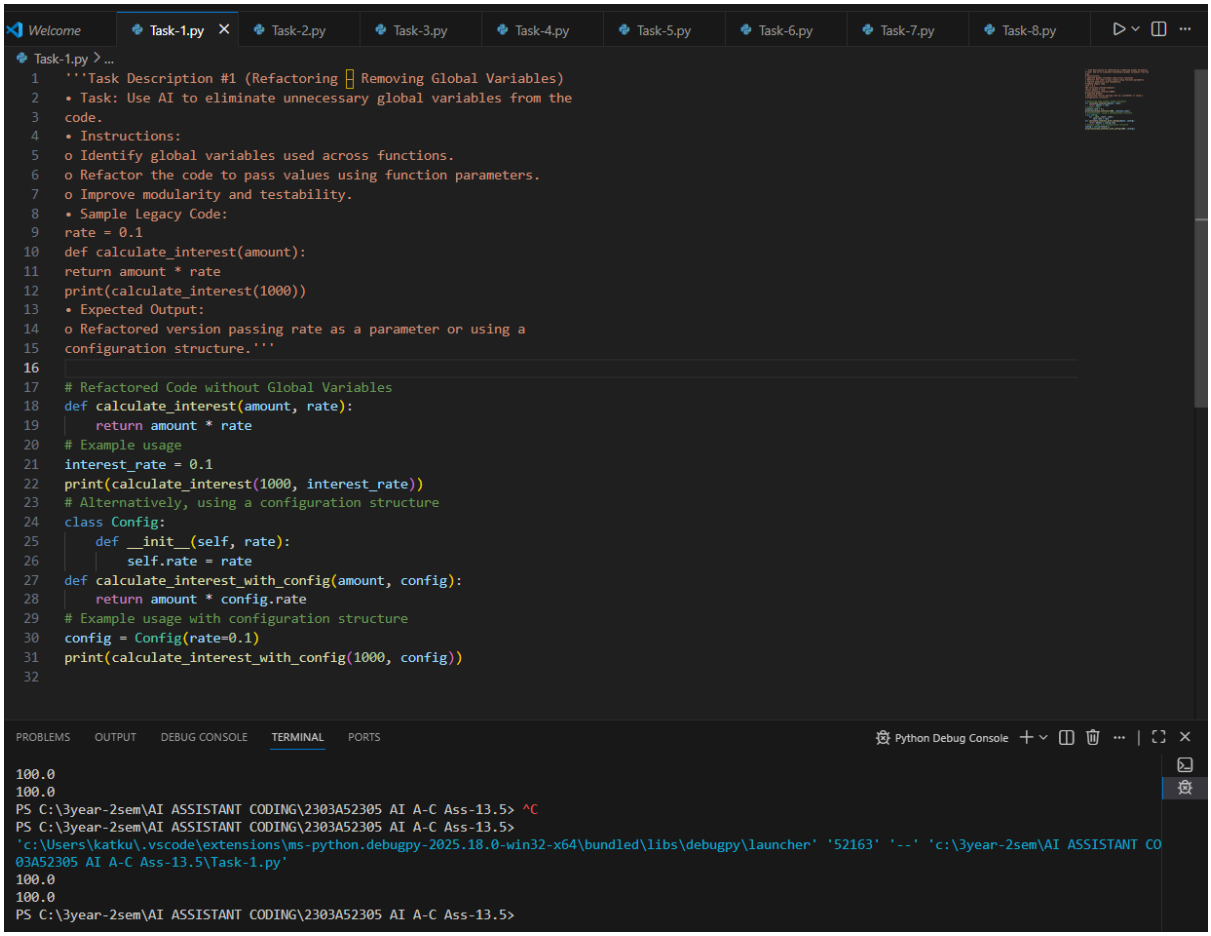
• Sample Legacy Code:

```
rate = 0.1

def calculate_interest(amount):

return amount * rate

print(calculate_interest(1000))
```

• Expected Output:

o Refactored version passing rate as a parameter or using a

configuration structure.




---

## Task Description #2 : (Refactoring Deeply Nested Conditionals)

• Task: Use AI to refactor deeply nested if–elif–else logic into a cleaner structure.

• Focus Areas:

o Readability

o Logical simplification

o Maintainability

Legacy Code:

score = 78

if score >= 90:

print("Excellent")

else:

if score >= 75:

print("Very Good")

else:

if score >= 60:

print("Good")

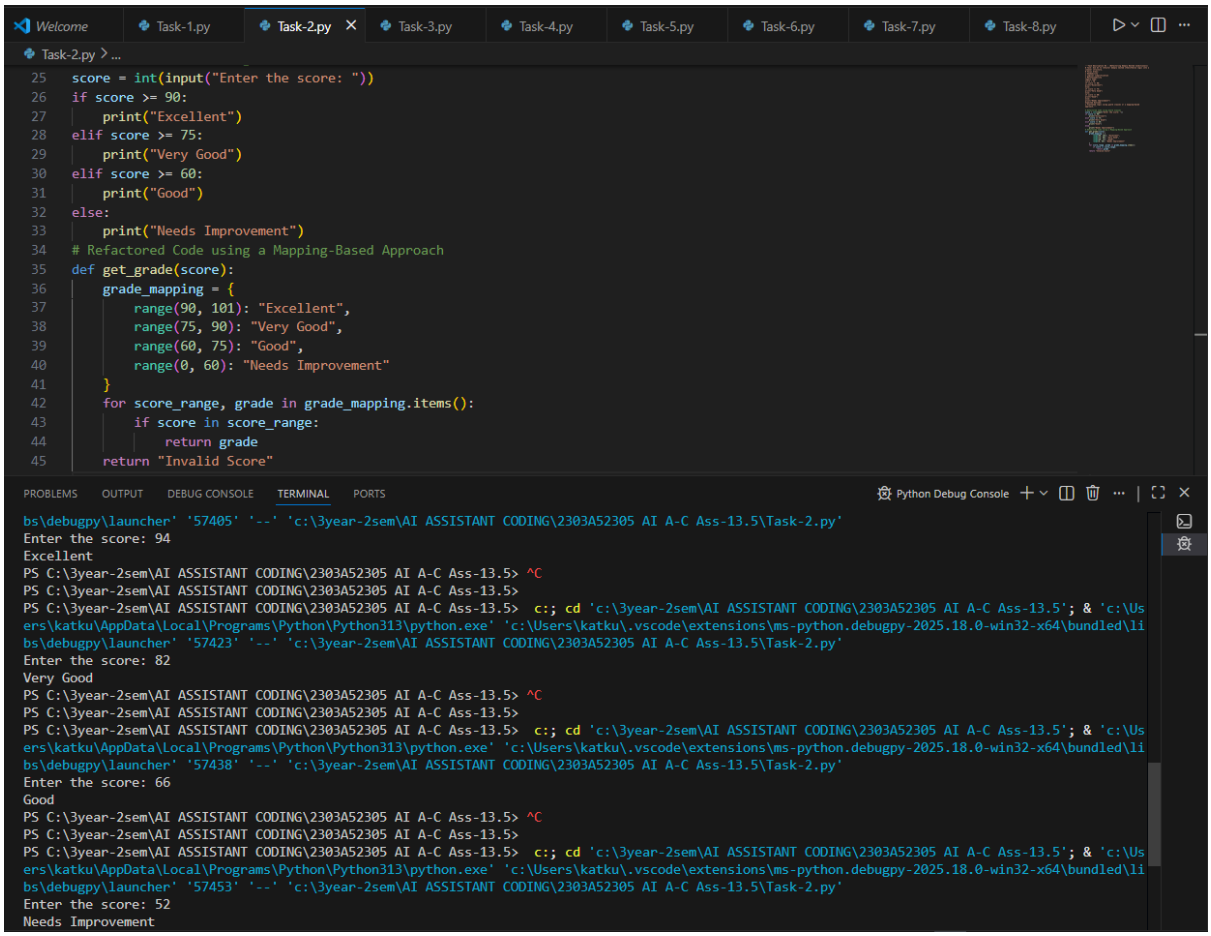
else:

print("Needs Improvement")

Expected Outcome:

o Flattened logic using guard clauses or a mapping-based

approach.




---

## Task 3 (Refactoring Repeated File Handling Code)

• Task: Use AI to refactor repeated file open/read/close logic.

• Focus Areas:

o DRY principle

o Context managers

o Function reuse

Legacy Code:

f = open("data1.txt")

print(f.read())

f.close()

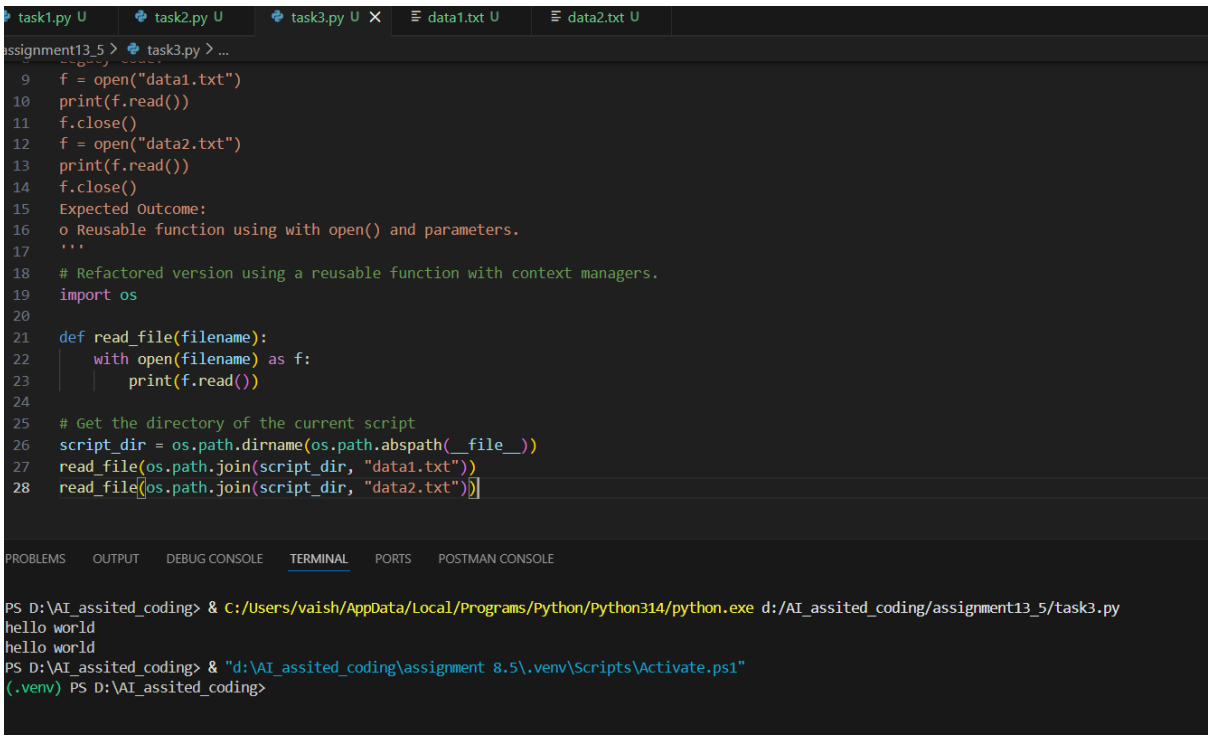
f = open("data2.txt")

print(f.read())

f.close()

Expected Outcome:

o Reusable function using with open() and parameters.




---

## Task 4 (Optimizing Search Logic)

• Task: Refactor inefficient linear searches using appropriate

data structures.

• Focus Areas:

o Time complexity

o Data structure choice

Legacy Code:

users = ["admin", "guest", "editor", "viewer"]

name = input("Enter username: ")

found = False

for u in users:

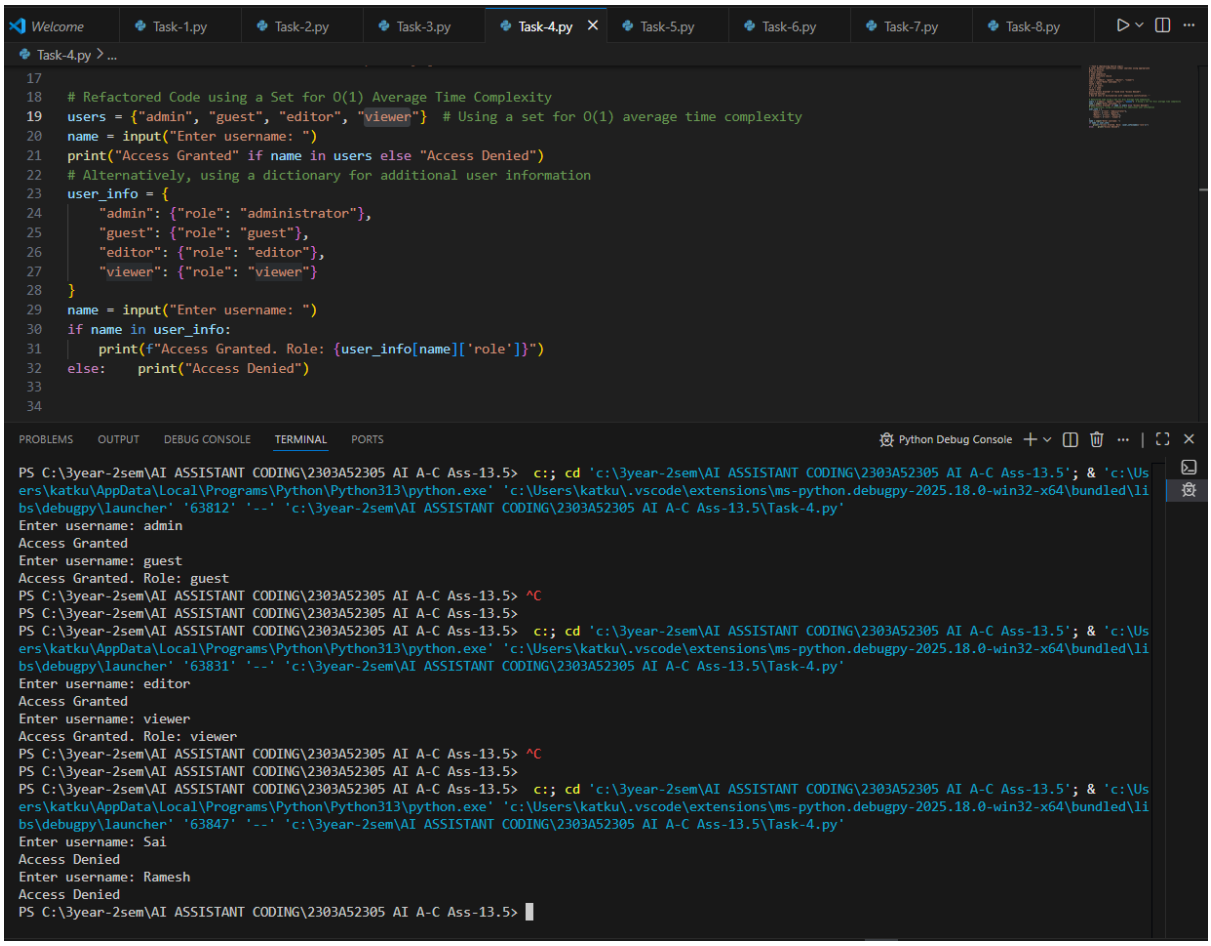
if u == name:

found = True

print("Access Granted" if found else "Access Denied")

Expected Outcome:

o Use of sets or dictionaries with complexity justification.



## Task 5 (Refactoring Procedural Code into OOP Design)

• Task: Use AI to refactor procedural code into a class-based design.

• Focus Areas:

o Object-Oriented principles

o Encapsulation

Legacy Code:

salary = 50000

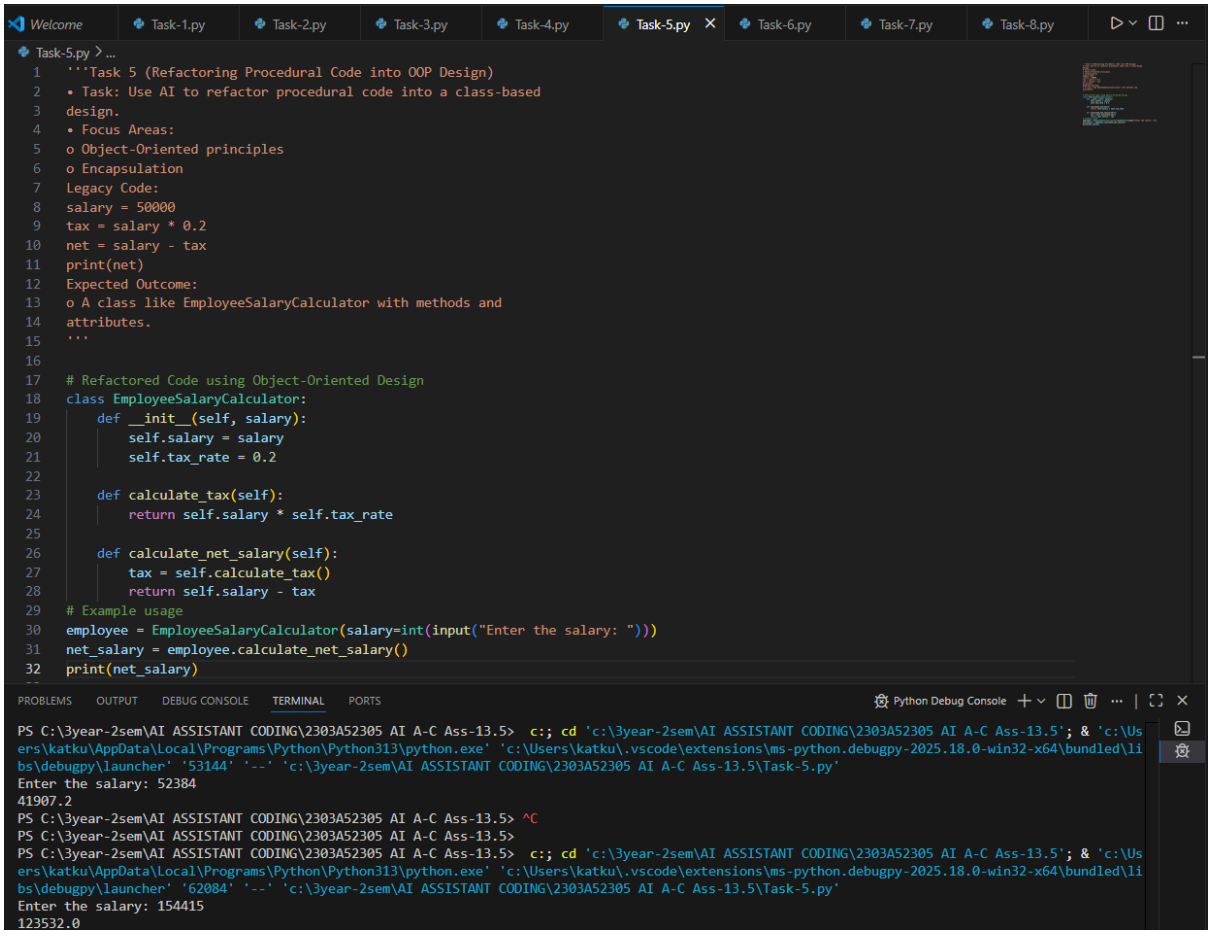
tax = salary * 0.2

net = salary - tax

print(net)

Expected Outcome:

o A class like EmployeeSalaryCalculator with methods and attributes.



## Task 6 (Refactoring for Performance Optimization)

• Task: Use AI to refactor a performance-heavy loop handling large data.

• Focus Areas:

o Algorithmic optimization

o Use of built-in functions

Legacy Code:

total = 0

for i in range(1, 1000000):

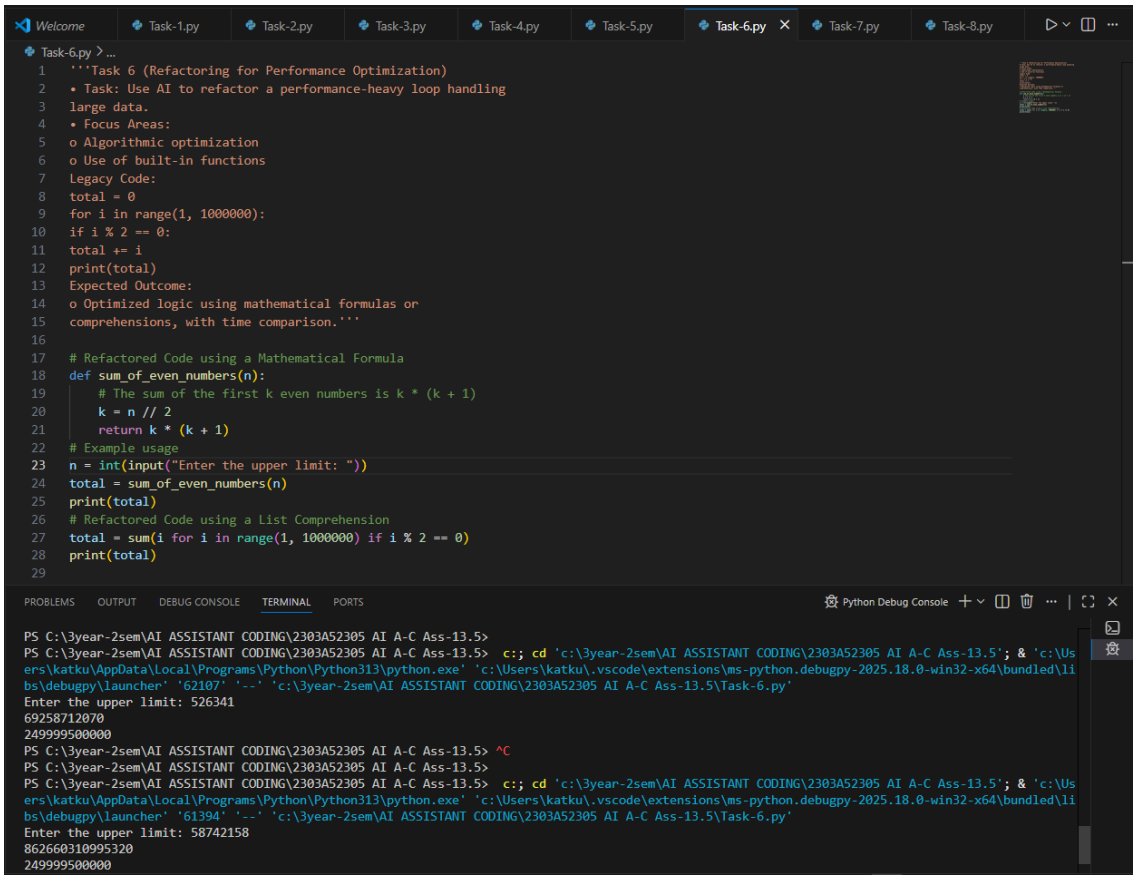
if i % 2 == 0:

total += i

print(total)

Expected Outcome:

o Optimized logic using mathematical formulas or comprehensions, with time comparison.



## Task 7 (Removing Hidden Side Effects)

• Task: Refactor code that modifies shared mutable state.

• Focus Areas:

o Functional-style refactoring

o Predictability

Legacy Code:

data = []

def add_item(x):

data.append(x)

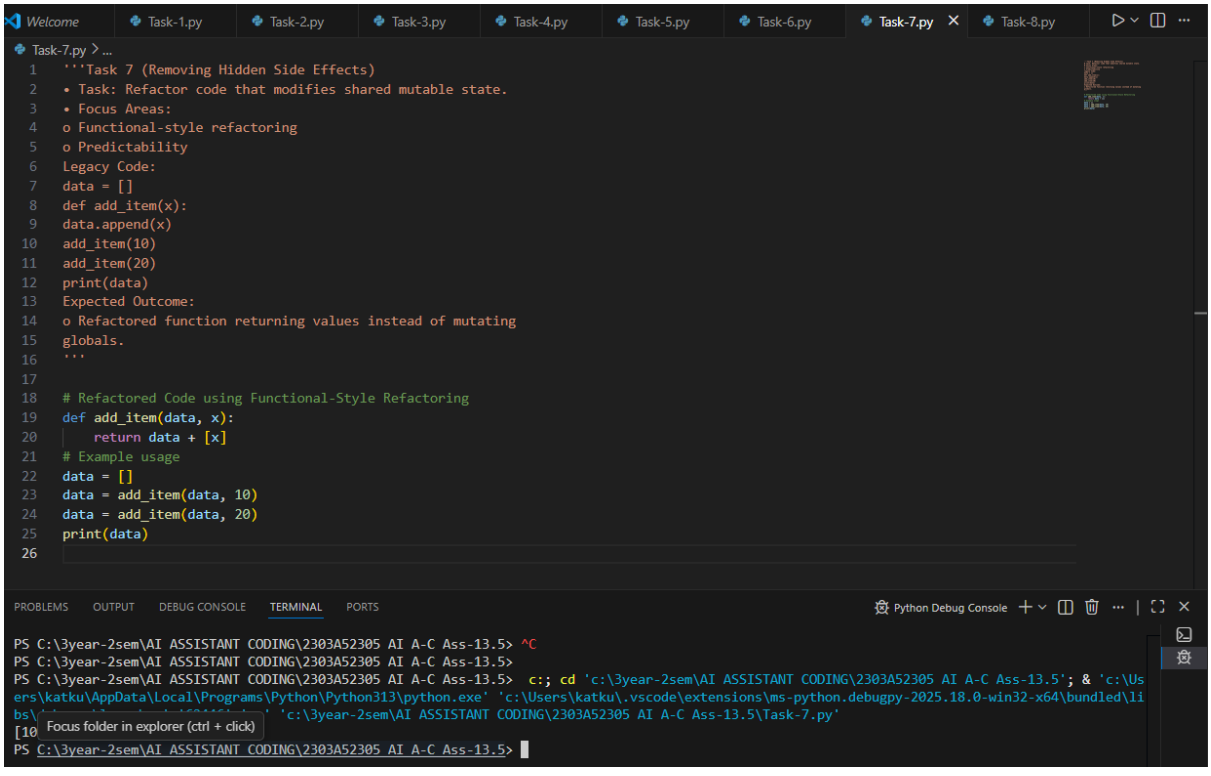
add_item(10)

add_item(20)

print(data)

Expected Outcome:

o Refactored function returning values instead of mutating globals.



**Task 8 (Refactoring Complex Input Validation Logic)**

• Task: Use AI to simplify and modularize complex validation

rules.

• Focus Areas:

o Readability

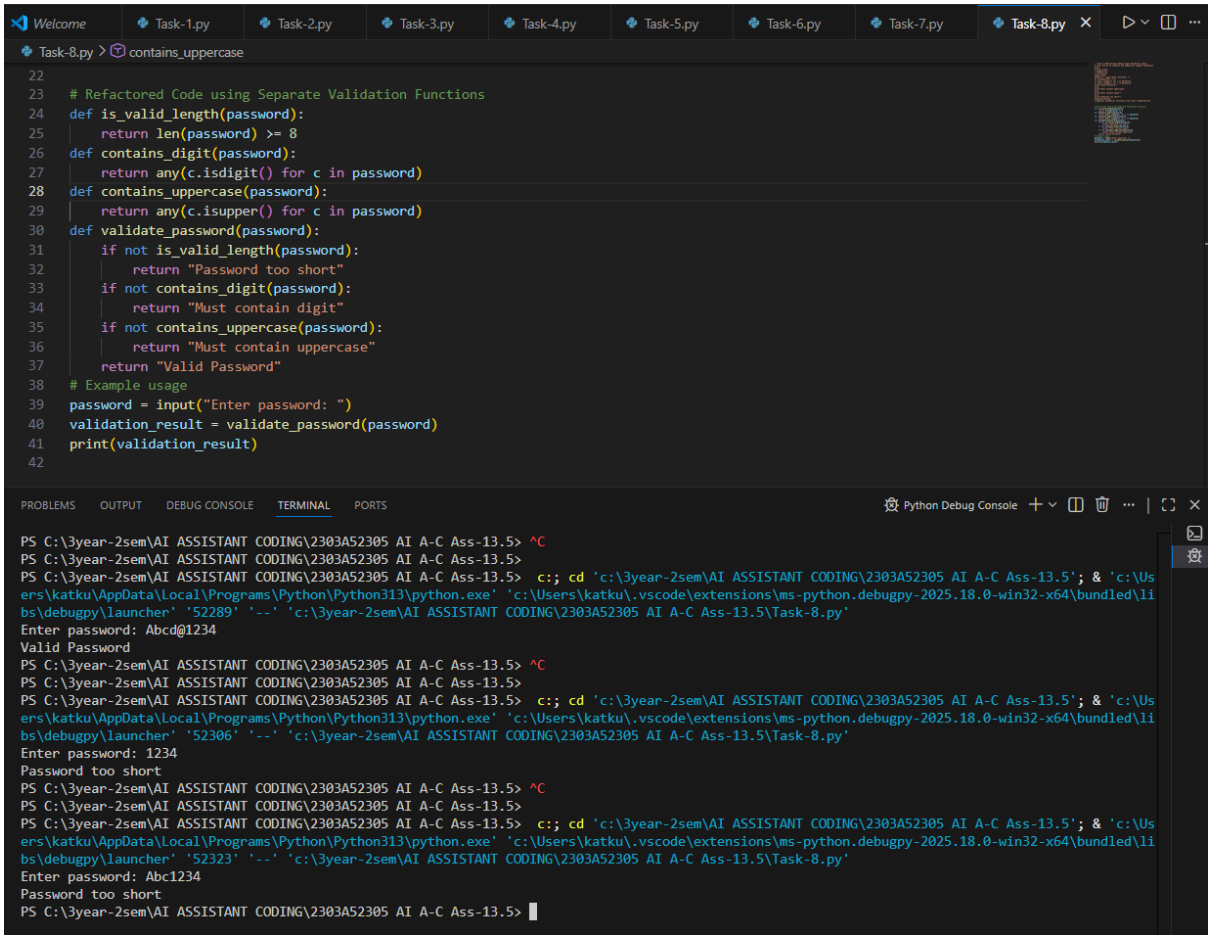
o Testability

Legacy Code:

```
password = input("Enter password: ")
if len(password) >= 8:
if any(c.isdigit() for c in password):
if any(c.isupper() for c in password):
print("Valid Password")
else:
print("Must contain uppercase")
else:
print("Must contain digit")
else:
print("Password too short")
```

Expected Outcome:

o Separate validation functions with clear responsibility


Welcome
Task-1.py
Task-2.py
Task-3.py
Task-4.py
Task-5.py
Task-6.py
Task-7.py
Task-8.py
Task-8.py > contains_uppercase
# Refactored Code using Separate Validation Functions
def is_valid_length(password):
return len(password) >= 8
def contains_digit(password):
return any(c.isdigit() for c in password)
def contains_uppercase(password):
return any(c.isupper() for c in password)
def validate_password(password):
if not is_valid_length(password):
return "Password too short"
if not contains_digit(password):
return "Must contain digit"
if not contains_uppercase(password):
return "Must contain uppercase"
return "Valid Password"
# Example usage
password = input("Enter password: ")
validation_result = validate_password(password)
print(validation_result)
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
Python Debug Console
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5> ^C
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5>
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5> c:; cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '52289' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5\Task-8.py'
Enter password: Abcd@1234
Valid Password
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5> ^C
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5>
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5> c:; cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '52306' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5\Task-8.py'
Enter password: 1234
Password too short
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5> ^C
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5>
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5> c:; cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '52323' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5\Task-8.py'
Enter password: Abc1234
Password too short
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5>